Name: V. Hansika pranavi
App No: cos/2025/1376
Sec: 4

DSA

1) Reverse a string using stack

* Algorithm

1. Start
2. Read the string
3. Push each character into stack
4. Pop characters one by one
5. Print popped characters [Reversed string]
6. Stop

C program

```
#include <stdio.h>
#include <string.h>
#define MAX 100
char stack [MAX];
int top = -1;
// Push function
void push (char ch){
    stack [++top]=ch;
}
// Pop function
char pop() {
    return stack [top--];
}
```

```
int main(){
    Char str[100];
    int i;
    printf ("Enter string:");
    scanf ("%s",str):
  // Push all characters
  for (i=0; i<strlen(str); i++){
        Push (str[i]);
    }
  printf ("Reversed string:");
  // pop to get reverse
    while (top! = -1){
        printf("%c",popc());
    }
    return 0;
}
```

Output:

Enter string : DATA

Reversed string : ATAD

2) Balanced Parentheses

Algorithm

1. Start
2. Read expression
3. Push opening brackets into stack
4. For closing brackets
   - check stack top
   - If matching → pop
   - Else → not balanced
5. If stack empty → balanced
6. stop

C program:

```
#include <stdio.h>
# include <string.h>
# define MAX 100
 char stack [MAX];
   int top = -1;
 // push bracket
 void push (char ch){
     stack [++top] = ch;

 }
 // pop bracket
  char pop ( ){
     return stack [top--];

}
```

```
int isMatchingPair(char open, char close){
    if (open == '(' && close == ')') return 1;
    if (open == '{' && close == '}') return 1;
    if (open == '[' && close == ']') return 1;
}

int main(){
    char exp[100];
    int i;
    printf("Enter expression:");
    scanf("%s", exp);
    for (i=0; i< strlen(exp); i++){
        if (exp[i]=='('||exp[i]=='{'||exp[i]=='[')
            push(exp[i]);
        else if (exp[i]==')'||exp[i]=='}'||exp[i]==']')
        {
            if (top==-1||!isMatchingPair(pop(),exp[i]))
            {
            printf("Not Balance Expression\n");
            return 0;
            }
        }
    }
}
    // check stack empty
    if (top==-1)
        printf("Balanced Expression\n");
    else
        printf("Not Balanced Expression\n");
    return 0;
}
```

Output:

Enter expression: (A+B) * (C-D)

Balanced expression

3) Next Greater Element

Algorithm

1. Start
2. Take array input
3. Traverse array
4. Use stack to track elements
5. For each element:
   - compare with stack top
   - print next greater
6. Stop

C program

```
# include <stdio.h>
# def MAX 100
int stack [MAX];
int top = -1;
// Push element
void Push (int x){
    stack [++top] = x;
}
// pop element
int pop() {
    return stack [top--];
}
```

```
int main() {
  int arr[]={4,5,2,10,8};
    int n=5,i;
  Push (arr[0]);
  for (i=1; i<n; i++){
    // find next greater
    while (top!=-1 && stack [top] <arr [i]){
      printf ("%d->%d\n", pop(),arr [i]);
    }
    Push (arr [i]);
  }
  // remaing elements
  while (top!=-1){
      printf ("%d ->-1\n", pop());
    }
    return 0;
  }
```

Out put:

4 ->5
5 ->10
2 ->10
10 ->-1
8 ->-1

4) Printer Queue simulation

Algorithm:

1. Start
2. Initialize queue
3. Display menu:
   - Add document
   - Print document
   - Display queue
4. Perform operations using FIFO
5. Stop

C Program

```
# include <stdio.h>
# define MAX 5
int queue[MAX];
int front=-1, rear=-1;
// Insert element
void enqueue(int x){
    if (rear==MAX-1){
        printf("Queue FULL\n");
        return;
    }
    if (front==-1) front=0;
    queue[++rear]=x;
}
```

```
// delete element
void dequeue() {
    if (front == -1){
        Printf ("queue Empty\n");
        return ;
    }
Printf ("printed document : %d\n", queue[front++]);
if (front > rear) front = rear = -1;
}
// display queue
void display() {
    int i;
    if (front == -1){
        Printf ("No pending documents\n");
        return;
    }
printf ("pending documents:");
for [i = front; i <= rear; i++)
    printf ("%d", queue[i]);
printf ("\n");
}
int main() {
int choice, doc;
while(1){
    Printf ("\n1.Add 2-Print 3.Display 4.Exit\n");
    Scanf ("%d", & choice);
```

```
Switch (choice){
    Case 1:
      Printf ("Enter document id:");
      Scanf ("%d", &doc);
      enqueue(doc);
      break;
    Case 2:
        dequeue();
        break;
    Case 3:
        display();
        break;
    Case 4:
        return 0;
    }
  }
}
```

Output:

```
1.Add 2.Print 3.Display 4.Exit
1
Enter document id : 103
1. Add 2.Print 3. Display 4.Exit
3
Pending documents: 103
```